

Proyecto final transversal

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional, disponible en <https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.



Universidad
Rey Juan Carlos



Máster Universitario en Análisis de Datos Deportivos

Práctica final

Big Data Processing II

**Sistema de análisis deportivo en tiempo real con
Kafka, Spark, LangGraph y generación
automática de informes enriquecidos mediante
RAG**

1. Contexto

Una entidad del ámbito deportivo desea construir un sistema capaz de monitorizar eventos de un partido en tiempo real y generar automáticamente un informe útil para analistas, cuerpo técnico y personal de comunicación.

El sistema debe ser capaz de:

- recibir eventos deportivos en tiempo real;
- procesarlos de forma distribuida;
- integrar datos adicionales relevantes;
- calcular métricas e indicadores de rendimiento;
- recuperar información contextual desde una base documental;
- generar un informe automático en lenguaje natural;
- coordinar el proceso final mediante un flujo de agentes implementado con LangGraph.

El objetivo de la práctica es desarrollar una solución aplicada en Python que combine tecnologías Big Data, procesamiento en streaming e inteligencia artificial generativa dentro de un caso de uso deportivo realista.

2. Objetivo general

Desarrollar un pipeline completo de analítica deportiva que procese eventos en tiempo real mediante **Kafka** y **Spark Structured Streaming**, y que produzca un informe automático final enriquecido mediante **RAG**, coordinando la recuperación y la generación mediante **LangGraph** y agentes con tools.

3. Caso de uso

Se trabajará sobre el escenario de **análisis en tiempo real de un partido de fútbol**.

Durante el partido se generan eventos como pases, tiros, goles, recuperaciones, pérdidas, faltas o sprints. Estos eventos son publicados en Kafka y consumidos por un pipeline de Spark. A partir de ellos, el sistema calcula métricas agregadas por equipo y jugador.

Una vez obtenidos los resultados analíticos, el sistema genera un informe automático del partido. Para mejorar su calidad, el informe no debe basarse únicamente en estadísticas numéricas, sino también en información recuperada desde una colección documental. Por ejemplo:

- descripciones de jugadores,
- perfiles de equipos,

- información táctica,
- resúmenes históricos,
- contexto sobre estilos de juego,
- documentación técnica sobre métricas deportivas.

La fase final del sistema deberá estar orquestada mediante un flujo de agentes en LangGraph, de forma que la recuperación de contexto y la elaboración del informe se realicen de manera estructurada y modular.

4. Requisitos de la práctica

La práctica deberá desarrollarse en Python e incluir obligatoriamente los siguientes componentes:

4.1. Ingesta de datos en tiempo real con Kafka

El sistema deberá incluir un productor que envíe eventos deportivos a un tópico de Kafka y un consumidor basado en Spark Structured Streaming que procese dichos eventos.

Cada evento deberá incluir, al menos, los siguientes campos:

- event_id
- timestamp
- match_id
- team
- player
- event_type
- zone
- minute
- value

El alumnado podrá partir de un conjunto de datos base o de un generador sintético de eventos.

4.2. Procesamiento distribuido con Spark Structured Streaming

El pipeline deberá consumir los eventos desde Kafka y realizar, al menos, las siguientes tareas:

- lectura del stream;
- definición explícita del esquema;
- validación y limpieza básica de datos;
- transformación de tipos;
- cálculo de métricas agregadas;
- generación de resultados analíticos intermedios y finales.

Se espera que el procesamiento se realice con PySpark, utilizando operaciones adecuadas para datos en streaming.

4.3. Integración de datos heterogéneos

Además del flujo principal de eventos, la solución deberá integrar al menos una fuente adicional de información estática o semiestructurada, por ejemplo:

- información de jugadores;
- alineaciones;
- zonas del campo;
- estadísticas históricas;
- catálogo de métricas;
- documentos descriptivos del contexto deportivo.

Esta integración deberá utilizarse para enriquecer el análisis o la generación del informe final.

4.4. Cálculo de métricas deportivas

El sistema deberá calcular métricas relevantes para el análisis del partido.

Como mínimo, deberán obtenerse indicadores por equipo y por jugador.

Métricas mínimas por equipo:

- número total de eventos;
- tiros;
- goles;
- faltas;
- recuperaciones;
- intensidad por intervalos temporales.
- Métricas mínimas por jugador
- participación total;
- eventos ofensivos;
- eventos defensivos;
- contribución acumulada al juego.

Se valorará que las métricas tengan sentido deportivo y estén correctamente justificadas.

4.5. Persistencia de resultados

Los resultados del pipeline deberán almacenarse de forma estructurada.

Como mínimo, deberán persistirse:

- los datos procesados o depurados;

- las métricas agregadas finales;
- el informe generado.

Se recomienda usar formatos como Parquet, CSV o una base de datos ligera, siempre que la elección esté justificada.

4.6. Optimización y escalabilidad

La práctica deberá incluir un apartado de optimización técnica del pipeline. El alumnado deberá aplicar y justificar al menos tres medidas relacionadas con eficiencia o escalabilidad, por ejemplo:

- filtrado temprano;
- selección de columnas necesarias;
- particionado;
- uso razonado de caché o persistencia;
- elección adecuada del formato de salida;
- control del tamaño de lotes o microbatches;
- simplificación de transformaciones.

No se exige una optimización avanzada de producción, pero sí una reflexión técnica coherente.

4.7. Generación automática de informe

A partir de las métricas calculadas, el sistema deberá generar un informe automático del partido en lenguaje natural.

El informe deberá incluir, como mínimo:

- resumen general del partido;
- equipo o jugador destacado;
- momento o tramo de mayor intensidad;
- interpretación básica del rendimiento observado;
- conclusión final orientada a un stakeholder no técnico.

4.8. Enriquecimiento mediante RAG

La generación del informe deberá apoyarse en un módulo de Retrieval-Augmented Generation (RAG).

Este módulo tendrá como objetivo recuperar información relevante desde una pequeña base documental y utilizarla para enriquecer el informe final.

La colección documental podrá contener, por ejemplo:

- descripciones de jugadores;
- estilos tácticos de equipos;

- resúmenes de encuentros anteriores;
- notas técnicas sobre indicadores de rendimiento;
- explicaciones de conceptos deportivos.

El alumnado deberá implementar un flujo básico de RAG que incluya:

- preparación o indexación de documentos;
- recuperación de contexto relevante ante una consulta;
- uso del contexto recuperado en el prompt final del modelo generativo.

El objetivo no es construir un sistema conversacional complejo, sino demostrar cómo una arquitectura RAG puede aportar contexto útil y verificable al informe generado.

Importante: el informe final deberá distinguir con claridad entre:

- información derivada de los datos del partido procesados en tiempo real;
- información contextual aportada por los documentos recuperados.

4.9. Orquestación con LangGraph y agentes con tools

La fase final de análisis y generación del informe deberá implementarse mediante LangGraph.

No será suficiente con realizar una única llamada directa a un modelo generativo. El alumnado deberá construir un flujo basado en nodos o agentes que coordine, al menos, las siguientes funciones:

- consulta de métricas generadas por el pipeline;
- recuperación de documentos relevantes del módulo RAG;
- elaboración del informe final a partir de la información disponible.

Para ello, deberán emplearse agentes con tools. Como mínimo, la solución deberá incluir:

- Tool 1. Consulta de métricas del partido
 - o Una herramienta que permita acceder a las métricas ya calculadas por Spark y almacenadas por el sistema.
 - o Ejemplos de consultas:
 - equipo con mayor número de recuperaciones;
 - jugador con mayor participación;
 - tramo temporal con más intensidad;
 - número total de tiros o faltas.
- Tool 2. Recuperación documental
 - o Una herramienta conectada al índice o base documental del módulo RAG para recuperar contexto relevante.
 - o Ejemplos:
 - descripción táctica de un equipo;

- perfil de un jugador;
- explicación de una métrica concreta;
- contexto histórico relacionado con el partido analizado.

También podrá plantearse una arquitectura con dos agentes especializados y un nodo final de consolidación.

Lo importante es que:

- exista una orquestación real en LangGraph;
- los agentes utilicen tools de forma explícita;
- el uso de agentes aporte estructura y trazabilidad al proceso;
- el resultado final esté conectado de forma coherente con el pipeline de Big Data.

No se exige un sistema multiagente excesivamente complejo, pero sí una implementación clara, modular y funcional.

5. Salida final del sistema

La práctica deberá producir una salida final comprensible y bien presentada. Esta salida podrá adoptar una de las siguientes formas:

- notebook con tablas, gráficos e informe final;
- aplicación sencilla en Streamlit;
- informe HTML o PDF generado automáticamente.

La salida deberá mostrar de forma clara:

- métricas obtenidas;
- elementos más relevantes del análisis;
- contexto recuperado desde la base documental;
- informe narrativo final generado mediante el flujo en LangGraph.

6. Tecnologías mínimas

La práctica deberá desarrollarse utilizando, como mínimo:

- Python
- Apache Kafka
- PySpark / Spark Structured Streaming
- LangGraph

Además, será necesario incorporar alguna solución para:

- indexación y recuperación documental;
- embeddings o búsqueda semántica;
- generación de texto mediante un modelo generativo.

El alumnado podrá elegir la tecnología concreta, siempre que quede correctamente explicada.

7. Entregables

7.1. Código fuente

El alumnado deberá entregar el código completo, organizado de forma clara.

La estructura recomendada es la siguiente:

```
final_project/
├── data/
│   ├── static/
│   └── docs/
├── output/
│   ├── processed/
│   ├── aggregates/
│   └── reports/
├── src/
│   ├── kafka_producer.py
│   ├── streaming_pipeline.py
│   ├── metrics.py
│   ├── rag_pipeline.py
│   ├── tools.py
│   ├── graph_workflow.py
│   ├── report_generator.py
│   └── app.py
├── requirements.txt
├── README.md
└── memoria.pdf
```

7.2. Memoria técnica

La memoria deberá describir de forma clara la solución implementada y deberá incluir, al menos:

- objetivo del proyecto;
- arquitectura general;
- descripción de los datos;

- uso de Kafka;
- procesamiento con Spark;
- métricas calculadas;
- diseño del módulo RAG;
- diseño del flujo en LangGraph;
- tools implementadas y función de cada una;
- generación del informe;
- decisiones de optimización;
- limitaciones del sistema.

7.3. Presentación oral

La práctica se defenderá oralmente en la fecha indicada oficialmente en la convocatoria de exámenes.

La presentación deberá mostrar:

- el problema abordado;
- la arquitectura desarrollada;
- una demostración del flujo completo;
- el papel de Kafka, Spark, RAG y LangGraph;
- los principales resultados;
- las decisiones técnicas adoptadas.

8. Criterios generales de valoración

En la evaluación se tendrá en cuenta:

- el correcto funcionamiento del pipeline completo;
- la integración adecuada de Kafka y Spark;
- la calidad del análisis deportivo realizado;
- la coherencia del módulo RAG dentro de la solución;
- la correcta utilización de LangGraph y agentes con tools;
- la calidad del informe automático generado;
- la justificación técnica de las decisiones adoptadas;

- la claridad de la memoria y de la presentación oral.

9. Consideraciones importantes

- La práctica debe funcionar de extremo a extremo.
- No se valorará positivamente incluir tecnologías de forma superficial.
- Kafka debe utilizarse como mecanismo real de ingesta en tiempo real.
- El RAG debe aportar contexto útil al informe final.
- LangGraph debe emplearse para coordinar de forma explícita la fase final del sistema.
- Los agentes deberán utilizar tools reales conectadas con los datos y documentos del proyecto.
- La solución debe estar orientada a un caso de uso deportivo realista.
- El código deberá ser legible, modular y reproducible.

10. Integridad académica

Todo el trabajo entregado debe ser original o estar correctamente referenciado. El uso de asistentes de IA deberá limitarse a tareas de apoyo y nunca podrá sustituir la comprensión técnica del proyecto ni la autoría efectiva del código y de la memoria.

MATERIAL DOCENTE EN ABIERTO